

$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

The problem of minimizing maximum weighted completion time on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a scheduling problem we establish to be strongly NP-hard. We first establish that the optimal solution maintains the permutation property and introduce a novel block structure where jobs are partitioned by distinct weights, enabling efficient dynamic programming solutions. The exact dynamic programming algorithm leverages interval-based state representation and runs in $O(n \cdot K \cdot V^{3K-3})$ time for constant K , where V is the total processing time. For approximation guarantees, we design a fully polynomial-time approximation scheme (FPTAS) with state-class labeling that achieves $(1 + \varepsilon)$ -approximation while maintaining polynomial complexity. Additionally, we develop practical heuristics including an adapted NEH algorithm, an ant colony optimization (ACO) algorithm (Zhang et al., 2023), and a genetic algorithm (Reeves, 1995) that provide fast solutions for large-scale instances. The computational results demonstrate that our algorithms efficiently solve both small and large instances while maintaining theoretical guarantees.

Keywords: Flow shop; Dynamic programming; Approximation algorithms;

1 Introduction

Manufacturing plants that process their orders through two consecutive stages face a recurring trade-off between throughput and priority. The two stages are usually unbalanced: the first feeds the second and runs close to its capacity. The order in which the jobs are released to the line therefore determines how long each of them waits. Sequencing the jobs is a production planning decision, and it has been studied since Johnson’s rule settled the

*Corresponding author. E-mail address: author@email.com

two-machine case for the makespan criterion (Johnson, 1954). The model, its variants and its solution methods have been surveyed extensively since (Pinedo, 2022).

The configuration is common in manufacturing and appears in a variety of process industries. A metal-parts plant machines components and then passes them to a coating or heat-treatment line; a foundry casts and then rolls; a print shop runs a press and then a bindery; a pharmaceutical plant granulates a powder and then compresses it into tablets. In each case a job must finish its first operation before its second can begin, and each machine can process one job at a time. The plants differ in what makes one order more urgent than another. A supplier working under a late-delivery clause loses money for every hour an order is overdue; a plant building to stock has capital tied up in work in process; a contract manufacturer must respect the priority its largest customer has bought. In all three cases a cost is attached to the time an individual order spends in the plant. That cost is attached to the order rather than to the plant as a whole.

That cost is what the criterion of this paper measures. The objective is the maximum weighted completion time, $WC_{\max}$, also called the weighted makespan. Each order j contributes the product $w_j C_j$, and the objective is the largest of these products. It is therefore determined by the most costly order rather than by the average, and the weight decides which order that is. Minimizing $WC_{\max}$ amounts to protecting the most costly order from delay. This is the decision a planner faces when a line is loaded beyond its capacity and not every order can be completed on time.

In scheduling theory the criterion has been studied in several machine environments. On a single machine, jobs are revealed over time and must be sequenced without knowledge of the future; for this online setting, best-possible algorithms are known for the maximum weighted completion time (Li, 2015). A single machine carrying one restricted rate-modifying activity has also been treated for the same criterion (Zuo et al., 2026). On identical parallel machines, randomized approximation schemes for the weighted makespan have been developed (Sun, 2024), and the criterion has been combined with the decision of which orders to accept (Lu et al., 2025). Maximum-type criteria have been studied in other forms as well. For a single machine, the maximum earliness/tardiness cost has been minimized when jobs may be rejected (Guo et al., 2026). In all of these settings, however, a job occupies one machine for its entire processing time. The interaction between two consecutive stages, which is what defines the flow shop, therefore does not arise.

The flow shop environment, by contrast, has been studied extensively but for different criteria. The variant that has attracted most theoretical attention is the proportionate flow shop, in which every machine spends the same time on a given job; it, the ordered flow shop and related models are surveyed in (Panwalkar et al., 2013). Its appeal is that it can often be reduced to a single-machine problem with the same criterion (Panwalkar and Koulamas, 2015), so that results proved for one machine carry over. Weighted objectives have been treated within it as well, but the weighted objective is the sum of the weighted completion

times rather than its maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the criterion that has been studied is the makespan, most recently together with the decision of which orders to reject (Shabtay and Gerstl, 2024).

The criterion and the machine environment are therefore each well established, but their combination is not, and the works closest to this paper show how far apart the two remain. In the criterion group, every setting reviewed above keeps each job on a single machine for its whole processing time. This holds for the online single-machine results (Li, 2015), the rate-modifying variant (Zuo et al., 2026), the parallel-machine approximation schemes (Sun, 2024) and the order-acceptance model (Lu et al., 2025). In the environment group, the weighted objective that has been treated is a sum rather than a maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the criterion treated is the makespan (Shabtay and Gerstl, 2024). The maximum weighted completion time in a two-machine flow shop appears in none of them.

The problem is not solved by combining these results, for two reasons. The reductions that make the proportionate flow shop tractable require the processing times to be proportionate. The general two-machine flow shop does not satisfy that assumption, so results proved for one machine do not carry over. More fundamentally, a total weighted completion time decomposes over the jobs. It can therefore be accumulated while a schedule is built, which is what allows a dynamic program to compare partial schedules with one another. A maximum does not decompose in this way: a partial schedule carries no information about its final value until it is complete. Optimizing the maximum weighted completion time in a two-machine flow shop therefore requires structural properties of a kind that the earlier work does not supply.

In this paper we study the problem $F2 \parallel WC_{\max}$. Four features distinguish it from the settings reviewed above: (i) two machines in series, so that each completion time depends on the idle time inherited from its predecessors; (ii) an objective that is a maximum over weighted completion times rather than their sum; (iii) general processing times, with no proportionate assumption; and (iv) no rejection, so that every job must be scheduled. Replacing the weighted sum by a weighted maximum, and dropping the proportionality assumption, would already put the problem beyond the single-machine reduction. We show that $F2 \parallel WC_{\max}$ is in fact strongly NP-hard (Theorem 3.2). The difficulty is structural rather than a matter of the instance size. Because the objective is determined by a single job, an exchange that shortens many completion times while lengthening that one does not improve the objective value. Because the jobs carry unequal weights, moreover, Johnson’s rule has no direct extension to the weighted case.

Our results follow from the structure of an optimal schedule. Since the objective is a maximum, jobs of equal weight are interchangeable with respect to it. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson’s order within each block, and blocks in non-increasing weight order. On this structure we

build three methods. An exact dynamic program runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V the total processing time. It is practical when the weights fall into few classes, as in the plants described above. A 2-approximation runs in $O(n \log n)$ time. Rounding the input of the dynamic program turns it into a fully polynomial-time approximation scheme that attains a $(1 + \varepsilon)$ -approximation for constant K . Because the exact methods remain exponential in K , we also develop three heuristics for large instances and report a computational study that compares them with the 2-approximation.

The rest of the paper is organized as follows. Section 2 fixes the notation and states the completion-time recurrence. Section 3 gives the mixed integer programming formulation and establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the 2-approximation and the approximation scheme. Section 5 describes the three heuristics. Section 6 reports the computational study, and Section 7 concludes.

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series. Each job J_j must be processed first on M_1 for a_j time units and then on M_2 for b_j time units, and preemption is not allowed. All processing times a_j and b_j are assumed to be positive integers. Each machine can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ specifies the processing sequence on both machines. For a schedule π , let $A_j = \sum_{h=1}^j a_{\pi(h)}$ and $B_j = \sum_{h=1}^j b_{\pi(h)}$ be the cumulative processing times on M_1 and M_2 after the first j jobs. The completion time of the job in position j on machine M_2 is

$$C_{\pi(j)} = \max_{1 \leq i \leq j} \left\{ \sum_{h=1}^i a_{\pi(h)} + \sum_{h=i}^j b_{\pi(h)} \right\}. \quad (1)$$

The objective is to find a schedule π that minimizes $WC_{\max}$. Throughout, $WC_{\max}^*$ denotes the optimal value of $F2 \parallel WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, the positions are indexed by $k = 1, \dots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is assigned to position k , and 0 otherwise;
- C_k^1 completion time of the job in position k on M_1 ;
- C_k^2 completion time of the job in position k on M_2 ;
- C_j completion time of job J_j on M_2 ;

- Z the maximum weighted completion time $WC_{\max}$;
- M a sufficiently large constant, taken as $2 \sum_{j=1}^n (a_j + b_j)$.

The MILP model is:

$$\min \quad Z \tag{2}$$

$$\text{s.t.} \quad \sum_{k=1}^n x_{jk} = 1, \quad j = 1, \dots, n \tag{3}$$

$$\sum_{j=1}^n x_{jk} = 1, \quad k = 1, \dots, n \tag{4}$$

$$C_1^1 = \sum_{j=1}^n a_j x_{j1} \tag{5}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^n a_j x_{jk}, \quad k = 2, \dots, n \tag{6}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^n b_j x_{j1} \tag{7}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^n b_j x_{jk}, \quad k = 2, \dots, n \tag{8}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^n b_j x_{jk}, \quad k = 2, \dots, n \tag{9}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad j = 1, \dots, n, \quad k = 1, \dots, n \tag{10}$$

$$Z \geq w_j C_j, \quad j = 1, \dots, n \tag{11}$$

$$x_{jk} \in \{0, 1\}, \tag{12}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{13}$$

Constraint (3)–(4) ensure a one-to-one assignment between jobs and positions. Constraints (5)–(6) compute the completion time of the job in each position on M_1 . Constraints (7)–(9) compute the completion time of the job in each position on M_2 . For $k \geq 2$, the job in position k starts on M_2 only after it has completed M_1 and the preceding job has released M_2 . Constraint (10) links the position-based completion times to the job-based completion times through a big- M formulation: when $x_{jk} = 1$, job J_j is in position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$ and the constraint is inactive. Constraint (11) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$. Constraint (12) requires the assignment variables x_{jk} to be binary. Constraint (13) requires the completion times C_k^1 , C_k^2 and C_j to be nonnegative.

3 Problem formulation

Theorem 3.1. $F2 \parallel WC_{\max}$ is NP-hard.

Proof. We reduce from the Partition problem: given r positive integers $x_1, \dots, x_r$ with $\sum_{i=1}^r x_i = 2T$, do there exist disjoint subsets $Z_1, Z_2 \subseteq \{1, \dots, r\}$ with $Z_1 \cup Z_2 = \{1, \dots, r\}$ and

$$\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T?$$

Construct an instance of $F2 \parallel WC_{\max}$ with $n = r + 1$ jobs. For $j = 1, \dots, n - 1$, set $a_j = 1$, $b_j = rx_j$ and $w_j = T + 1$; for the remaining job J_n , set $a_n = rT$, $b_n = 1$ and $w_n = 2T + 1$. The decision question is whether some schedule satisfies

$$WC_{\max} \leq Y, \quad \text{where} \quad Y = r(T + 1)(2T + 1).$$

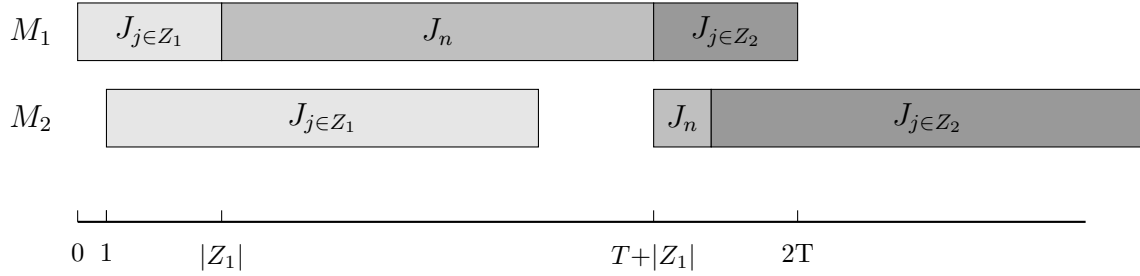


Figure 3-1

The jobs $J_1, \dots, J_{n-1}$ are partitioned into two subsets by J_n ; the resulting layout on the two machines is shown in Figure 3-1. The total M_2 processing times of the two subsets are unknown, but their sum equals $2rT$. We may therefore write them as $r(T + \xi)$ and $r(T - \xi)$ for some $-T \leq \xi \leq T$. The completion times of J_n are then

$$C_n^1 = |Z_1| + rT + 1, \tag{14}$$

$$C_n^2 = 1 + r(T + \xi) + 1, \tag{15}$$

and $C_n = \max\{C_n^1, C_n^2\}$.

If $\xi = 0$, then $\sum_{j \in Z_1} b_j = \sum_{j \in Z_2} b_j$, so the Partition instance has a feasible solution. In this case C_n is given by (14), and

$$\begin{aligned} C_n &= |Z_1| + rT + 1 \leq rT + r = r(T + 1), \\ w_n C_n &= (2T + 1)r(T + 1) = Y, \\ WC_{\max} &= r(T + 1)(C_n + rT) = Y, \end{aligned}$$

so $w_j C_j \leq WC_{\max} = Y$.

Conversely, suppose that some schedule satisfies $WC_{\max} \leq Y$. When $\xi > 0$, C_n is given by (15), and

$$\begin{aligned} C_n &= 1 + r(T + \xi) + 1 \geq 1 + r(T + 1) + 1, \\ w_n C_n &\geq (2T + 1)(1 + r(T + 1) + 1) = Y. \end{aligned}$$

When $\xi < 0$, C_n is given by (14), and

$$\begin{aligned} C_{\max} &= C_n + \sum_{j \in Z_2} b_j \\ &= 1 + rT + |Z_1| + r(T - \xi) \\ &\geq rT + r + rT \\ &= r(2T + 1) = Y, \\ WC_{\max} &\geq r(T + 1)(r(2T + 1)) = Y, \end{aligned}$$

so $WC_{\max} > Y$, a contradiction.

The Partition instance therefore admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. $\square$

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \dots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B ?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \dots, J_{3m}$ and m separator jobs $S_1, \dots, S_m$:

- for every element job J_j , set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k , set $a_{S_k} = RB + 1$, $b_{S_k} = 1$.

All processing times are positive integers. For $k = 1, \dots, m$, define $d_k = kD + 1$ and let $Q = (T + 2)^3$. Set

$$w_{S_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \quad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \dots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \leq T + 1$ and $Q \geq d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \dots, d_m, T\}$ and every integer completion time C ,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \leq Q \implies C \leq d. \quad (16)$$

Indeed, if $C \geq d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \leq r < d$. Then $q(d + 1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \dots, G_m$ be triples with $\sum_{j \in G_k} x_j = B$ for every k , and schedule

$$G_1, S_1, G_2, S_2, \dots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1 , and S_k uses $RB + 1$ units. Hence S_k completes on M_1 at time kD . The three element jobs in G_k use $RB + 3$ units on M_2 ; since the preceding separator completes on M_2 at time $(k - 1)D + 1$, the last element of G_k completes at time kD . Therefore

$$C_{S_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q , so $WC_{\max} \leq Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \leq Y$. Since the objective is regular, the standard permutation-schedule property for two-machine flow shops lets us restrict attention to permutation schedules. The separator jobs have identical processing times, so if their positions are fixed, exchanging the labels of two separators that occur in the wrong order can only decrease the maximum weighted completion time, because $w_{S_1} \geq w_{S_2} \geq \dots \geq w_{S_m}$. We may consequently assume that the separators occur in the order $S_1, S_2, \dots, S_m$. By (16),

$$C_{S_k} \leq kD + 1 \quad (k = 1, \dots, m), \quad C_j \leq T \quad (j = 1, \dots, 3m).$$

No element job can occur after S_m . Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing sequences on M_1 and M_2 coincide, so this job is also the last job on M_1 . All m separators and all $3m$ element jobs therefore complete on M_1 no later than it, so its M_1 completion time is at least the total M_1 processing time

$$m(RB + 1) + 3m = m(RB + 4) = T.$$

Its M_2 completion time would then be strictly greater than T , contradicting $C_j \leq T$. The element jobs are therefore partitioned into the m groups G_k lying between S_{k-1} and S_k , where S_0 denotes the start of the schedule. Put

$$B_k = \sum_{j \in G_k} x_j, \quad E_k = \sum_{h=1}^k |G_h|, \quad E_0 = 0.$$

For $k = 2, \dots, m$, the M_2 path gives

$$C_{S_k} \geq C_{S_{k-1}} + RB_k + |G_k| + 1, \tag{17}$$

while the M_1 path to S_{k-1} gives

$$C_{S_{k-1}} \geq (k-1)(RB+1) + E_{k-1} + 1. \quad (18)$$

Combining (17) and (18) with $C_{S_k} \leq kD+1$ yields

$$R(B_k - B) + E_k \leq 3k, \quad k = 2, \dots, m. \quad (19)$$

For $k = 1$, the M_2 path and $C_{S_1} \leq D+1$ similarly give

$$R(B_1 - B) + E_1 \leq 4. \quad (20)$$

Because $R = 3m + 5$, (19) and (20) imply $B_k \leq B$ for every k . Otherwise $B_k \geq B + 1$, and the respective left-hand side exceeds $3k$, or exceeds 4 when $k = 1$. Since every element job belongs to one of the groups, $\sum_{k=1}^m B_k = mB$, and hence $B_k = B$ for every k . Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B , while at least four elements have sum strictly greater than B . Thus $G_1, \dots, G_m$ is a valid 3-partition.

The construction uses $4m$ jobs, and all processing times, weights and the threshold are bounded by a polynomial in m and B . Since 3-Partition remains NP-complete when B is polynomially bounded in m , this is a strong polynomial reduction, and $F2 \parallel WC_{\max}$ is strongly NP-hard. $\square$

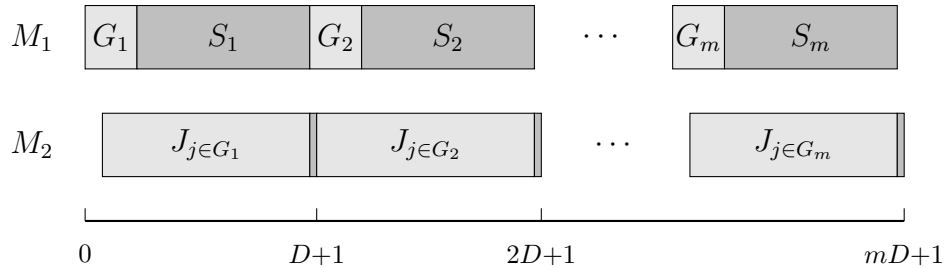


Figure 3-2: 3-Partition reduction

4 Algorithm

In this section, we design a dynamic programming algorithm for $F2 \parallel WC_{\max}$ and analyze its running time. Throughout, let K denote the number of distinct weights and $V = \sum_{j=1}^n (a_j + b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is thus fixed once we decide how each weight class is distributed

among the blocks. This is what makes a dynamic program possible, and why the running time of every algorithm below is governed by K .

We build three algorithms on this structure. The first algorithm is exact, and it prunes dominated states. A state records each block's processing time on both machines, the completion time of its last job on M_2 , and that job's weight. It runs in $O(n \cdot K \cdot |\mathcal{H}|_{\max})$ time, with $|\mathcal{H}|_{\max} \leq 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$ time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1 + \epsilon)$ -approximation scheme for a fixed number of distinct weights.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same order (Baker, 1974); we therefore restrict attention to such permutation schedules. Within this class, Johnson's rule (Johnson, 1954) minimizes the makespan: the jobs with $a_j \leq b_j$ are scheduled first in non-decreasing order of a_j , and the remaining jobs follow in non-increasing order of b_j .

Fix a permutation schedule π . Let $W_1 > W_2 > \dots > W_K$ be the K distinct weights occurring among the jobs, and let

$$\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \quad k = 1, \dots, K,$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \dots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that is, among $\pi(\ell_{k-1} + 1), \dots, \pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1} + 1), \dots, \pi(\ell_k)\}, \quad k = 1, \dots, K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1), \dots, \pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1, \dots, \mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^k \mathcal{B}_i = \{\pi(\ell_k + 1), \dots, \pi(n)\}, \quad k = 1, \dots, K.$$

Hence $\ell_1 \leq \ell_2 \leq \dots \leq \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2 , respectively, C_i the completion time of its last job on M_2 , and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k . By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k , whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \dots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. Consider a six-job instance with sequence $\pi = (J_1, \dots, J_6)$ and the following (a_j, b_j, w_j) values:

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, and $\mathcal{B}_3 = \{J_6\}$.

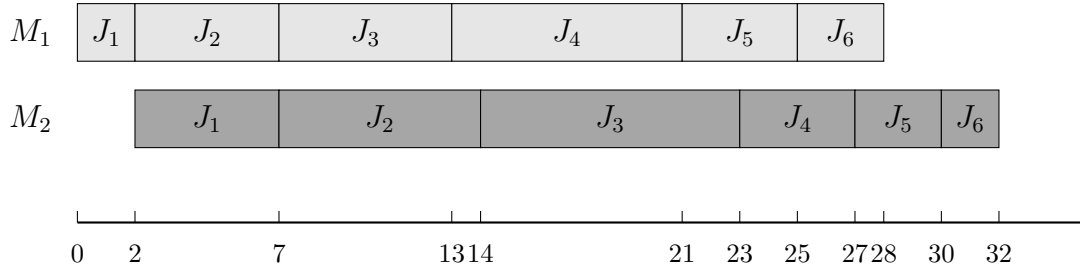


Figure 4-1

Lemma 4.1. For any schedule π , rescheduling the jobs within each block according to Johnson's rule (Johnson, 1954) and concatenating the blocks in their original order yields a schedule π' with $WC_{\max}(\pi') \leq WC_{\max}(\pi)$.

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k , and every job in $\mathcal{B}_k$ has weight at most L_k : a weight occurring in $\mathcal{B}_k$ either equals W_k , or it has already appeared in an earlier block, whose weights are larger. Let $C_k(\sigma)$ denote the completion time of the last job of $\mathcal{B}_k$ under schedule σ . An empty block satisfies $L_k C_k(\sigma) = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j , so $C_j(\sigma) \leq C_k(\sigma)$, while $w_j \leq L_k$ by the block construction. Hence $w_j C_j(\sigma) \leq L_k C_k(\sigma)$, and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j(\sigma) = L_k C_k(\sigma).$$

The objective therefore equals

$$WC_{\max}(\sigma) = \max_{k=1, \dots, K} L_k C_k(\sigma),$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_k(\pi') \leq C_k(\pi)$ for every k by induction on k .

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_1(\pi') \leq C_1(\pi)$.

Inductive step. Assume $C_{k-1}(\pi') \leq C_{k-1}(\pi)$. On M_1 , the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \dots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2 , block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^k A_i, C_{k-1}\}$; the first term is identical under π and π' , and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π . Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_k(\pi') \leq C_k(\pi)$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_k(\pi') \leq \max_k L_k C_k(\pi) = WC_{\max}(\pi),$$

which establishes the lemma. $\square$

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes the jobs sequentially in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \dots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \dots, n$, let $\mathcal{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i : the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i : the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i : the completion time of the last job of block $\mathcal{B}_i$ on machine M_2 ;
- L_i : the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \dots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{i : w_j \leq W_i\},$$

the eligible set is $\mathcal{E}(J_j) = \{1, \dots, i^*\}$. Each state is therefore the aggregate K -block tuple

$$(A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K),$$

which is described by $4K$ parameters.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathcal{H} = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$.

1 **Step 1.** [Initialization] Set $T_{A_0} \leftarrow 0, T_{B_0} \leftarrow 0, WC_{\max} \leftarrow 0$.

2 **Step 2.** [Evaluation] **for** $i = 1$ **to** K **do**

3 $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$

4 $T_{B_i} \leftarrow \max\{T_{A_{i-1}} + C_i, T_{B_{i-1}} + B_i\}$

5 **if** $L_i > 0$ **then**

6 $WC_{\max} \leftarrow \max\{WC_{\max}, L_i \cdot T_{B_i}\}$

7 **end**

8 **end**

Output: $WC_{\max}$.

The initial state set contains a single state whose components are all zero: $\mathcal{H}_0 = \{(0, \dots, 0; 0, \dots, 0; 0, \dots, 0; 0, \dots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2 .

Lemma 4.2. *Let $\mathcal{H} = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$ and $\mathcal{H}' = (A'_1, \dots, A'_K; B'_1, \dots, B'_K; C'_1, \dots, C'_K; L'_1, \dots, L'_K)$ be two states in $\mathcal{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i , and $C_i \leq C'_i$ for every i , then $\mathcal{H}'$ can be discarded without affecting the optimal solution.*

Proof. Consider any continuation that extends $\mathcal{H}'$ to a complete schedule Π' . Replacing the prefix represented by $\mathcal{H}'$ with the prefix represented by $\mathcal{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^k A_i, C_{k-1}\}$, and C_{k-1} is no larger under $\mathcal{H}$ than under $\mathcal{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathcal{H}$. Since A_i , B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π' . $\square$

For $j = 1, \dots, n$, the algorithm extends $\mathcal{H}_{j-1}$ to $\mathcal{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathcal{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i -th slice of the state is updated while all other blocks remain unchanged:

$$A'_i = A_i + a_j, \quad B'_i = B_i + b_j, \quad C'_i = \max\{C_i + b_j, A_i + a_j + b_j\}. \quad (21)$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \dots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathcal{H}_j$.

To support Step 4, each state in $\mathcal{H}_j$ stores a pointer to the state in $\mathcal{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathcal{H}_n$ attaining Z^* , backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j . Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) ; block count K ; weights $W_1 > \dots > W_K$.

```
1 Step 1. [Initialization] Set  $L_i \leftarrow 0$  for  $i = 1, \dots, K$ , set  
    $\mathcal{H}_0 \leftarrow \{(0, \dots, 0; 0, \dots, 0; 0, \dots, 0; 0, \dots, 0)\}$  and  $Z^* \leftarrow +\infty$ .  
2 Step 2. [Generation] for  $j = 1$  to  $n$  do  
3    $\mathcal{Y}_i \leftarrow \emptyset$  for  $i = 1, \dots, K$   
4   for each  $(A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K) \in \mathcal{H}_{j-1}$  do  
5     for each block  $i \in \mathcal{E}(J_j) = \{1, \dots, i^*\}$  do  
6        $A'_i \leftarrow A_i + a_j$ ,  $B'_i \leftarrow B_i + b_j$ ,  $L'_i \leftarrow w_j$   
7        $C'_i \leftarrow \max\{C_i + b_j, A_i + a_j + b_j\}$   
8       Add the state  
          $(A_1, \dots, A'_i, \dots, A_K; B_1, \dots, B'_i, \dots, B_K; C_1, \dots, C'_i, \dots, C_K; L_1, \dots, L'_i, \dots, L_K)$   
         to  $\mathcal{Y}_i$   
9     end  
10    [Elimination]  $\mathcal{H}_j \leftarrow \text{ELIMINATE}(\mathcal{Y}_1, \dots, \mathcal{Y}_K)$  by Lemma 4.2  
11  end  
12 end  
13 Step 3. [Calculate] for each state  
    $S = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K) \in \mathcal{H}_n$  do  
14   if  $L_i \in \{0, W_i\}$  for every  $i = 1, \dots, K$  then  
15      $WC_{\max} \leftarrow$  Algorithm 1 applied to  $S$   
16      $Z^* \leftarrow \min\{Z^*, WC_{\max}\}$   
17   else  
18     Discard  $S$ .  
19   end  
20 end  
21 Step 4. [Result] Recover a schedule attaining  $Z^*$  by standard backtracking.  
   Output: An optimal schedule  $\sigma$ .
```

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O(n \cdot K \cdot |\mathcal{H}|_{\max})$ time, where $|\mathcal{H}|_{\max} \leq 2^K V^{3K-3}$ and $V = \sum_{j=1}^n a_j + \sum_{j=1}^n b_j$. For constant K , this is $O(nV^{3K-3})$.*

Proof. Each state in $\mathcal{H}_j$ is the $4K$ -tuple

$$(A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0, V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence, $\sum_{i=1}^K A_i$ equals the sum of the a_h over that prefix. At most $K-1$ of the A_i are therefore independent, contributing at most V^{K-1} distinct A -vectors. Symmetrically, $\sum_{i=1}^K B_i$ is

the prefix sum of the b_j , so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \leq A_i + B_i \leq V$, the C -vector contributes at most V^{K-1} distinct values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L -vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathcal{H}_j| \leq 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j , the algorithm takes each state in $\mathcal{H}_{j-1}$, tries every block $i \in \{1, \dots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \dots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathcal{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathcal{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathcal{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K , 2^K is constant and the bound becomes $O(n \cdot V^{3K-3})$. $\square$

4.3 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \leq 2 WC_{\max}^*$.*

Proof. Let σ be the schedule returned by Algorithm 3, let j_k be the job in position k of σ , and let $S_k = \{j_1, \dots, j_k\}$. Since the jobs are sorted by non-increasing weight, every job of S_k has weight at least w_{j_k} .

We first bound the completion times in σ . By (1), the job in position k completes on M_2 at time $C_{j_k}(\sigma)$, and each term in the maximum is bounded by the full sum:

$$\sum_{h=1}^i a_{j_h} + \sum_{h=i}^k b_{j_h} \leq \sum_{h=1}^k a_{j_h} + \sum_{h=1}^k b_{j_h} = \sum_{h \in S_k} (a_h + b_h), \quad 1 \leq i \leq k.$$

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) .

- 1 **Step 1.** Sort the jobs by non-increasing weight w_j (ties broken arbitrarily).
- 2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule σ .

Hence

$$C_{j_k}(\sigma) \leq \sum_{h \in S_k} (a_h + b_h). \quad (22)$$

Since $WC_{\max}(\sigma) = \max_{1 \leq k \leq n} w_{j_k} C_{j_k}(\sigma)$, multiplying (22) by $w_{j_k} > 0$ and taking the maximum over k gives

$$WC_{\max}(\sigma) = \max_{1 \leq k \leq n} w_{j_k} C_{j_k}(\sigma) \leq \max_{1 \leq k \leq n} w_{j_k} \sum_{h \in S_k} (a_h + b_h). \quad (23)$$

It remains to bound the right-hand side by $2WC_{\max}^*$. Fix a prefix S_k and let π^* be an optimal schedule. Let u be the job of S_k that finishes last on M_1 in π^* , and v the job of S_k that finishes last on M_2 . All jobs of S_k are processed on M_1 before u completes M_1 , so $C_u^* \geq \sum_{h \in S_k} a_h$; likewise, all M_2 processing of the jobs in S_k precedes v 's completion on M_2 , so $C_v^* \geq \sum_{h \in S_k} b_h$. Since $u, v \in S_k$, we have $w_u, w_v \geq w_{j_k}$. Because $WC_{\max}^*$ bounds the weighted completion time of every job in π^* , the bound $C_u^* \geq \sum_{h \in S_k} a_h$ together with $w_u \geq w_{j_k}$ gives

$$WC_{\max}^* \geq w_u C_u^* \geq w_{j_k} \sum_{h \in S_k} a_h, \quad (24)$$

and, similarly, $C_v^* \geq \sum_{h \in S_k} b_h$ together with $w_v \geq w_{j_k}$ gives

$$WC_{\max}^* \geq w_v C_v^* \geq w_{j_k} \sum_{h \in S_k} b_h. \quad (25)$$

Combining (24) and (25) yields

$$WC_{\max}^* \geq w_{j_k} \max \left\{ \sum_{h \in S_k} a_h, \sum_{h \in S_k} b_h \right\} \geq \frac{w_{j_k}}{2} \left(\sum_{h \in S_k} a_h + \sum_{h \in S_k} b_h \right), \quad (26)$$

where the last inequality uses $\max\{x, y\} \geq (x+y)/2$. Hence (26) gives $w_{j_k} \sum_{h \in S_k} (a_h + b_h) \leq 2WC_{\max}^*$ for every k , and the claim follows from (23). $\square$

4.4 FPTAS

4.4.1 Rounding the time coordinates

We now convert the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by rounding the input data (Hall, 1994). One key point of this technique is the determination of valid lower and upper bounds for the coordinates to be rounded: each time coordinate ranges in a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a single factor θ . The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2 , and its completion time C_i , are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \quad V_A = \sum_{j=1}^n a_j, \quad V_B = \sum_{j=1}^n b_j, \quad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (21) preserves $A_i \leq V_A$ and $B_i \leq V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \leq A_i + B_i \leq V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \geq 1 \text{ with } \theta^k \leq x < \theta^{k+1}, \quad \text{and} \quad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \geq 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \leq \theta y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathcal{H}^\epsilon = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$ is the tuple formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathcal{H}^\epsilon) = (\Delta(A_1), \dots, \Delta(A_K); \Delta(B_1), \dots, \Delta(B_K); \Delta(C_1), \dots, \Delta(C_K); L_1, \dots, L_K).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ , where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

The Elimination step keeps one representative per label. At iteration j , every state of $\mathcal{H}_{j-1}^\epsilon$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1, \dots, i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1, \dots, L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \leq \theta \tilde{x}$ in every time coordinate $x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathcal{H}}^\epsilon$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathcal{H}_j^\epsilon$, and the number of states never exceeds the number of labels.

Lemma 4.3. *Let $\mathcal{H}_j = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$, $j = 1, \dots, n$, be the states along an optimal trajectory of Algorithm 2, that is, the states recovered by backtracking from an optimal final state. For every j , Algorithm 4 keeps a state $\hat{\mathcal{H}}_j^\epsilon = (\hat{A}_1, \dots, \hat{A}_K; \hat{B}_1, \dots, \hat{B}_K; \hat{C}_1, \dots, \hat{C}_K; L_1, \dots, L_K)$ that carries the same block-end weights $L_1, \dots, L_K$ and satisfies*

$$\hat{A}_i \leq A_i \theta^{j-1}, \quad \hat{B}_i \leq B_i \theta^{j-1}, \quad \hat{C}_i \leq C_i \theta^{j-1}, \quad (1 \leq i \leq K). \quad (27)$$

Proof. We argue by induction on j , from 1 to n .

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) ; block count K ; accuracy ϵ .

- 1 **Step 1.** [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
 $\mathcal{H}_0^\epsilon \leftarrow \{(0, \dots, 0; 0, \dots, 0; 0, \dots, 0; 0, \dots, 0)\}$.
 - 2 **Step 2.** [Generation] **for** $j = 1$ **to** n **do**
 - 3 $\mathcal{Y}_i \leftarrow \emptyset$ **for** $i = 1, \dots, K$
 - 4 **for each state** $(A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K) \in \mathcal{H}_{j-1}^\epsilon$ **do**
 - 5 **for each block** $i \in \mathcal{E}(J_j) = \{1, \dots, i^*\}$ **do**
 - 6 $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
 - 7 $\tilde{C}_i = \max\{C_i + b_j, A_i + a_j + b_j\}$
 - 8 [Label] Record the label $\ell(\mathcal{H}_j^\epsilon) =$
 $(\Delta(\tilde{A}_1), \dots, \Delta(\tilde{A}_K); \Delta(\tilde{B}_1), \dots, \Delta(\tilde{B}_K); \Delta(\tilde{C}_1), \dots, \Delta(\tilde{C}_K); \tilde{L}_1, \dots, \tilde{L}_K)$.
 - 9 **end**
 - 10 [Elimination] Among the candidate states sharing the same label, keep one of them
 and delete the others; set $\mathcal{H}_j^\epsilon$ to the kept states.
 - 11 **end**
 - 12 **end**
 - 13 **Step 3.** [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathcal{H}_n^\epsilon$.
 - 14 **Step 4.** [Result] Same as Step 4 of Algorithm 2.
- Output:** An approximate schedule σ .
-

Base case $j = 1$. Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathcal{H}_1^\epsilon$ and $\mathcal{H}_1$ therefore coincide, and (27) holds with equality at $j = 1$.

Inductive step $j \geq 2$. Assume that (27) holds at $j - 1$, and let i be the block to which $\mathcal{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \leq W_i$, so $w_j \leq W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\mathcal{H}_{j-1}^\epsilon$ as well. Write A'_i , B'_i and C'_i for the coordinates of $\mathcal{H}_{j-1}$. Applying the exact recurrences to $\mathcal{H}_{j-1}^\epsilon$ and using $\theta^{j-2} \geq 1$ gives a candidate $\mathcal{H}_j^\epsilon$ with

$$\begin{aligned}\tilde{A}_i &= \hat{A}_i + a_j \leq (A'_i + a_j)\theta^{j-2} = A_i\theta^{j-2}, \\ \tilde{B}_i &= \hat{B}_i + b_j \leq (B'_i + b_j)\theta^{j-2} = B_i\theta^{j-2}, \\ \tilde{C}_i &= \max\{\hat{C}_i + b_j, \hat{A}_i + a_j + b_j\} \\ &\leq \max\{C'_i + b_j, A'_i + a_j + b_j\}\theta^{j-2} = C_i\theta^{j-2},\end{aligned}$$

where A_i , B_i and C_i are the coordinates of $\mathcal{H}_j$. For the remaining blocks $m \neq i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \leq x'_m\theta^{j-2} = x_m\theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \leq x_m\theta^{j-2}$ holds for every block $m = 1, \dots, K$. The block-end weights $L_1, \dots, L_K$ of the candidate are those of $\mathcal{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathcal{H}}_j^\epsilon$ carries the label $\ell(\tilde{\mathcal{H}}_j^\epsilon)$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathcal{H}}_j^\epsilon \in \mathcal{H}_j^\epsilon$ with that same label. Since the label records the block-end weights, $\hat{\mathcal{H}}_j^\epsilon$ carries $L_1, \dots, L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ , so $y_m^x \leq \theta \tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1, \dots, K$, where y_m^x denotes the coordinates of $\hat{\mathcal{H}}_j^\epsilon$. Together with $\tilde{x}_m \leq x_m \theta^{j-2}$, this yields (27) at j and completes the induction.

Every state of $\mathcal{H}_n^\epsilon$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (27): a derived value would accumulate the rounding errors of the remaining blocks. $\square$

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a fully polynomial-time approximation scheme with*

$$WC_{\max} \leq (1 + \epsilon) WC_{\max}^*.$$

For fixed K , the algorithm runs in $O\left(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\right)$ time.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathcal{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathcal{H}}_n^\epsilon \in \mathcal{H}_n^\epsilon$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathcal{H}}_n^\epsilon$ are within a factor θ^{n-1} of those of $\mathcal{H}_n$, and its block-end weights are those of $\mathcal{H}_n$, because the weights are not rounded. The final state $\mathcal{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathcal{H}}_n^\epsilon$ has block-end weight at most W_i .

Let $\hat{\sigma}$ and σ^* be the schedules obtained by concatenating the blocks of $\hat{\mathcal{H}}_n^\epsilon$ and $\mathcal{H}_n$, and let $C_i(\sigma)$ denote the completion time of the last job of block $\mathcal{B}_i$ in a schedule σ . By the concatenation recurrence of Algorithm 1, $C_i(\sigma)$ is nondecreasing in every coordinate of the state and positively homogeneous. Hence $C_i(\hat{\sigma}) \leq \theta^{n-1} C_i(\sigma^*)$ for every i . Since every block of $\hat{\mathcal{H}}_n^\epsilon$ has block-end weight at most W_i , Lemma 4.1 yields

$$WC_{\max}(\hat{\sigma}) = \max_{1 \leq i \leq K} W_i C_i(\hat{\sigma}) \leq \theta^{n-1} \max_{1 \leq i \leq K} W_i C_i(\sigma^*) = \theta^{n-1} WC_{\max}^*.$$

The factor θ^{n-1} is at most $1 + \epsilon$ for $0 < \epsilon < 1$. Because $1 + x \leq e^x$ for every real x ,

$$\theta^{n-1} = \left(1 + \frac{\epsilon}{2n}\right)^{n-1} \leq e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \leq 1 + \epsilon$ on $[0, 1]$. The function $g(\epsilon) = 1 + \epsilon - e^{\epsilon/2}$ satisfies $g(0) = 0$ and

$$g'(\epsilon) = 1 - \frac{1}{2}e^{\epsilon/2} \geq 1 - \frac{1}{2}e^{1/2} > 0 \quad (0 \leq \epsilon \leq 1),$$

because $e^{\epsilon/2} \leq e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon) \geq 0$ and $e^{\epsilon/2} \leq 1 + \epsilon$. Consequently $\theta^{n-1} \leq 1 + \epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\sigma}$, has a value of at most $(1 + \epsilon) WC_{\max}^*$.

We now bound the running time. At each level j , Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A + 2)^K (v^B + 2)^K (v^C + 2)^K (n + 1)^K$. The rounded time coordinates contribute $(v^A + 2)^K (v^B + 2)^K (v^C + 2)^K$, which is at most $(v^C + 2)^{3K}$ since $v^A, v^B \leq v^C$, and each of the K block-end weights takes at most $n + 1$ values. Each state generates at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathcal{H}^{\epsilon, n}$ costs $O(K)$ per state. The running time is therefore $O(nK (v^C + 2)^{3K} (n + 1)^K)$. Since $\ln(1 + \epsilon/(2n)) \geq \epsilon/(4n)$ for $0 < \epsilon < 1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln \theta} \right\rceil = O\left(\frac{n \log V}{\epsilon}\right).$$

For constant K , the bound reduces to $O(n^{4K+1} K (\log V)^{3K} / \epsilon^{3K})$, which is polynomial in n , $\log V$ and $1/\epsilon$. $\square$

4.4.2 Weight rounding

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \leq c$ for a constant c . Partition the interval $[W_{\min}, W_{\max}]$ into

$J_0 = [W_{\min}, W_{\min}\theta)$, $J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1})$ ($1 \leq r \leq q - 1$), $J_q = [W_{\min}\theta^q, W_{\max}]$, with $q = \lceil \log_\theta(W_{\max}/W_{\min}) \rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min} \theta^{r_j+1}, \quad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \leq \hat{w}_j \leq \theta w_j$. The rounded weights take only $q + 1 = O(\log_\theta(W_{\max}/W_{\min}))$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\sigma}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C_{\max}^*$. Then*

$$WC_{\max}(\hat{\sigma}) \leq \theta WC_{\max}^*. \quad (28)$$

Proof. By construction $w_j \leq \hat{w}_j \leq \theta w_j$ for every job J_j . Since the maximum is monotone, every schedule σ satisfies

$$WC_{\max}(\sigma) = \max_{1 \leq j \leq n} w_j C_j(\sigma) \leq \max_{1 \leq j \leq n} \hat{w}_j C_j(\sigma) = \hat{W}C_{\max}(\sigma) \leq \theta WC_{\max}(\sigma).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\sigma}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\sigma}) \leq \hat{W}C_{\max}(\hat{\sigma}) = \hat{W}C_{\max}^* \leq \hat{W}C_{\max}(\pi^*) \leq \theta WC_{\max}(\pi^*) = \theta WC_{\max}^*,$$

which is (28). $\square$

Hence the weight rounding incurs a relative error of at most $\theta = 1 + \epsilon/(2n) \leq 1 + \epsilon$, exactly as in the completion-time rounding.

5 Heuristic Algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. For large instances, fast heuristics are therefore of practical interest. In this section we present three heuristics for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Puka et al., 2024), an ant colony optimization (ACO) algorithm (Zhang et al., 2023), and a genetic algorithm (Reeves, 1995).

Throughout this section, the objective of a permutation $\sigma = (\sigma(1), \dots, \sigma(k))$ of k jobs is evaluated by (1), so that

$$WC_{\max}(\sigma) = \max_{1 \leq t \leq k} w_{\sigma(t)} C_{\sigma(t)},$$

which is computed in $O(k)$ time.

5.1 NEH

Constructive heuristics are attractive for $F2 \parallel WC_{\max}$ because they produce a complete schedule in a single pass, without the tuning effort of population-based metaheuristics. The NEH heuristic (Puka et al., 2024) is among the most effective constructive rules for permutation flow shops, and its core idea is to build a schedule by incrementally inserting jobs into a growing partial sequence, each time choosing the position that best serves the objective.

In the maximum weighted completion-time objective, two properties are particularly important. First, jobs with large total processing time $a_j + b_j$ tend to push the whole makespan forward and therefore should appear early so as not to delay other jobs. Second, given a fixed makespan, jobs with larger weight w_j benefit more from occupying earlier positions, because their weighted completion time decreases. These two properties suggest a greedy priority in which large- $a_j + b_j$ jobs are placed first and large- w_j jobs are used as a tie-breaker.

We adapt the NEH framework as follows. The jobs are first sorted by non-increasing $a_j + b_j$, breaking ties by non-increasing w_j , to obtain an initial sequence $J_1, \dots, J_n$. The schedule starts with the single job J_1 . For each $k = 2, \dots, n$, the next job J_k is inserted into every one of the k possible positions of the current partial schedule. Each candidate sequence is evaluated by (1), and the position that yields the smallest $WC_{\max}$ is retained. The pseudo-code is given in Algorithm 5.

Because the partial schedule already contains $k - 1$ jobs, the cost of evaluating all k insertions at step k is $O(k^2)$ using the recurrence (1); the total running time is therefore $O(n^3)$ (Theorem 5.1). The heuristic also produces a feasible permutation schedule by construction, requires no hyperparameter selection, and can serve either as a fast standalone solver for large instances or as a warm-start seed for the population-based heuristics described below.

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) .

- 1 **Step 1.** [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j ; let $J_1, \dots, J_n$ be the resulting sequence.
 - 2 **Step 2.** [Initialization] $\sigma \leftarrow (J_1)$.
 - 3 **Step 3.** [Insertion] **for** $k = 2$ **to** n **do**
 - 4 $\sigma \leftarrow$ the sequence obtained by inserting J_k into σ at a position minimizing $WC_{\max}$.
 - 5 **end**
- Output:** Schedule σ .
-

Theorem 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Proof. At step k , the job J_k is inserted into each of the k positions of a partial schedule of $k - 1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O(\sum_{k=1}^n k^2) = O(n^3)$. $\square$

5.2 Ant Colony Optimization

Population-based metaheuristics can explore a much larger set of schedules than constructive heuristics, but their effectiveness depends on how well the search bias matches the problem structure. Ant colony optimization (Zhang et al., 2023) is a constructive population metaheuristic in which a colony of artificial ants incrementally builds schedules, and the collective experience is stored in pheromone trails that guide subsequent ants toward promising job orderings.

For $F2 \parallel WC_{\max}$, the relative order of jobs is more important than an absolute assignment: placing a heavy job too late inflates the objective dramatically, while keeping it early usually helps regardless of the exact surrounding permutation. This makes pairwise preference information a natural search bias. ACO captures such preferences through a pheromone matrix τ_{ij} , where a larger value encourages job J_j to be scheduled after job J_i , thereby biasing the colony toward front-heavy schedules.

We adapt the standard ACO framework as follows. Initially all pheromone values are set to $\tau_{ij} = 1/n$. In every iteration, each of the m ants constructs a permutation σ_k by repeatedly selecting, from the set U of unscheduled jobs, the job J_j to be placed at the next position with probability

$$p_{jt} = \frac{\tau_j^\alpha \cdot \eta_{jt}^\beta}{\sum_{l \in U} \tau_l^\alpha \cdot \eta_{lt}^\beta}.$$

Here α and β control the relative influence of pheromone and heuristic information η_{jt} , which we set to $\eta_{jt} = w_j/(a_j + b_j + 1)$ so that jobs with large weight and small total processing time are preferred. After all ants have built their schedules, the best schedule σ^* found so

Algorithm 6: Ant Colony Optimization for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) ; parameters α, β, ρ, Q ; number of ants m .

```
1 Step 1. [Initialization] Initialize pheromone matrix  $\tau_{ij} \leftarrow 1/n$  for all  $i, j$ ; set best solution  $\sigma^*, Z^* \leftarrow +\infty$ .
2 Step 2. [Construction] for ant  $k = 1$  to  $m$  do
3   Build a permutation  $\sigma_k$  by probabilistically selecting jobs:
4   for position  $t = 1$  to  $n$  do
5     Calculate probabilities  $p_{jt} = \frac{\tau_j^\alpha \eta_{jt}^\beta}{\sum_{l \in U} \tau_l^\alpha \eta_{lt}^\beta}$  where  $U$  is unselected jobs;
6     Select job  $j$  with probability  $p_{jt}$  and set  $\sigma_k(t) = j$ ;
7   end
8   Evaluate  $Z_k = WC_{\max}(\sigma_k)$ ;
9   if  $Z_k < Z^*$  then
10     $\sigma^* \leftarrow \sigma_k, Z^* \leftarrow Z_k$ 
11  end
12 end
13 Step 3. [Update Pheromone] for each edge  $(i, j)$  do
14    $\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \rho \cdot \Delta\tau_{ij}$  where  $\Delta\tau_{ij} = Q/Z^*$  if  $(i, j)$  in  $\sigma^*$ , else 0;
15 end
16 Step 4. [Termination] if stopping criterion not met then
17   go to Step 2.
18 end
19
Output: Best schedule  $\sigma^*$ .
```

far is identified, and the pheromone trails are updated according to

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \rho \Delta\tau_{ij},$$

where $\rho \in (0, 1)$ is the evaporation rate and $\Delta\tau_{ij} = Q/Z^*$ if the edge (i, j) appears in σ^* , and 0 otherwise. The process is repeated until a stopping criterion is met. The pseudo-code is given in Algorithm 6.

Because ants construct complete schedules probabilistically, ACO can escape the restricted search region of deterministic heuristics. The pheromone mechanism also provides a compact, increasingly focused memory of good pairwise orders, so the algorithm naturally intensifies around high-quality regions while the stochastic construction preserves diversification. Moreover, each schedule is evaluated in $O(n)$ time by (1), making each iteration computationally lightweight and well suited to large instances.

5.3 Genetic Algorithm

The objective $\max_j w_j C_j$ is a bottleneck criterion: the value of a schedule is determined by the single *critical* job that attains the maximum, and an ordering can be improved only by lowering that job’s completion time, or by shortening completion times across the board. A genetic algorithm for $F2 \parallel WC_{\max}$ should therefore not treat all jobs symmetrically. Following the genetic algorithm (Reeves, 1995) for flow-shop sequencing, we design its operators around the weight classes and the block structure of Section 4: high-weight jobs should occupy early positions (their weighted completion must be kept small), while the makespan should stay low so that no job is delayed unnecessarily.

A chromosome is a permutation $\sigma = (\sigma(1), \dots, \sigma(n))$, where the gene at position t is the job scheduled t -th. Permutation encoding is adopted because it always represents a feasible permutation schedule: every job occurs exactly once, so no repair operator is needed to restore feasibility after crossover or mutation. To decode a chromosome, both machines process the jobs in the order $\sigma(1), \dots, \sigma(n)$; the completion time $C_{\sigma(t)}$ of the t -th job is obtained from the closed form (1), which is evaluated in $O(n)$ time using the prefix sums $A_t = \sum_{h=1}^t a_{\sigma(h)}$ and $B_t = \sum_{h=1}^t b_{\sigma(h)}$, and the objective value is

$$WC_{\max}(\sigma) = \max_{1 \leq t \leq n} w_{\sigma(t)} C_{\sigma(t)}.$$

The fitness $\text{fitness}(\sigma) = 1/WC_{\max}(\sigma)$ is used only to compare individuals, so its absolute scale is irrelevant.

Random starting populations waste most of the search effort. The initial population P_0 of size N is therefore seeded with the three orderings that already realize the two structural ingredients of a good schedule—small makespan and early heavy jobs:

- the Johnson schedule σ^J produced by Johnson’s rule (Johnson, 1954), which minimizes the makespan and thereby tends to shorten every completion time;
- the weight-descending schedule σ^W , obtained by ordering jobs by non-increasing w_j with ties broken by Johnson’s rule, which directly attacks the bottleneck by giving heavy jobs the smallest possible completion times;
- the schedule σ^{NEH} returned by the NEH heuristic (Algorithm 5).

The remaining $N - 3$ individuals are uniform random permutations, which supply the diversity needed to escape these deterministic starting points and to explore regions of the search space that no constructive rule reaches. The population size is $N = \Theta(n)$: large enough to retain the block structure present in the seeds, yet small enough that a generation, whose cost is dominated by evaluating N individuals in $O(n)$ time each, remains $O(Nn)$.

Binary tournament selection is used to form the mating pool. In each round, two individuals are drawn uniformly at random from the current population P_{g-1} , the fitter of the

Algorithm 7: Partially Mapped Crossover (PMX) for $F2 \parallel WC_{\max}$

Input: Parents π_1, π_2 ; cut points $p_1 < p_2$.

```
1 Step 1. [Copy segment] Set  $\pi'[t] \leftarrow \pi_1[t]$  for  $t = p_1, \dots, p_2$ .
2 Step 2. [Fill outer positions] for  $t = 1$  to  $n$ ,  $t \notin [p_1, p_2]$  do
3    $j \leftarrow \pi_2[t]$ 
4   while  $j$  occurs in  $\pi'$  do
5      $j \leftarrow \pi_2[\pi_1^{-1}(j)]$  // follow the PMX mapping
6   end
7    $\pi'[t] \leftarrow j$ 
8 end
```

Output: Child π' .

two wins and is copied into the mating pool M_g , and both are returned to the population (ties are broken arbitrarily); this is repeated until M_g contains N individuals. Tournament selection is preferred over roulette-wheel selection because it depends only on the *ranking* of fitness values rather than on their absolute magnitudes, so it remains selective even as the population converges and the fitness values of near-optimal schedules become very close.

Crossover recombines two parent schedules into offspring that inherit both the relative order and the absolute positions of jobs—the two ingredients a permutation schedule is built from. We use the partially mapped crossover (PMX), the standard operator for permutation-encoded genetic algorithms. Two cut points $p_1 < p_2$ are drawn uniformly at random, and the middle segment $\pi_1[p_1..p_2]$ is copied verbatim into the child. The remaining positions are filled from the second parent π_2 ; whenever a gene of π_2 already occurs in the child, it is replaced through the mapping induced by the middle segment, so the child is always a valid permutation. PMX transfers contiguous blocks of jobs, which realize the front-heavy structure identified in Section 4, while still recombining the two parents' orders on the outer positions. Swapping the roles of π_1 and π_2 yields the second child.

Mutation aims at the job that currently decides the objective. Let j^* be the critical job attaining $WC_{\max}(\sigma)$ at position t^* . Moving j^* to an earlier position $r < t^*$ can only decrease its own completion time (its machine-2 work is unchanged while it may start earlier), so the weighted completion $w_{j^*}C_{j^*}$ —currently the largest—tends to decrease. The mutation therefore tries every insertion of j^* into a position $r < t^*$, evaluates the resulting permutation by (1), and keeps the best; this is a cheap, targeted hill-climbing step. If no such insertion improves the value, a random swap of two jobs from different weight classes is performed with probability p_m to maintain diversity. Applied to every offspring, and combined with the elitist replacement, this operator acts as a local search that monotonically improves the incumbent without being trapped by the deterministic seeds.

Algorithm 8: Critical-Job Mutation for $F2 \parallel WC_{\max}$

Input: Schedule σ ; mutation probability p_m .

- 1 **Step 1.** [Identify] Evaluate $WC_{\max}(\sigma)$ by (1); let j^* be a job attaining the maximum and let t^* be its position in σ .
- 2 **Step 2.** [Insertion search] **for** $r = 1$ **to** $t^* - 1$ **do**
- 3 Form $\sigma^{(r)}$ by moving j^* to position r ; evaluate $WC_{\max}(\sigma^{(r)})$.
- 4 **end**
- 5 **Step 3.** **if** $\min_r WC_{\max}(\sigma^{(r)}) < WC_{\max}(\sigma)$ **then**
- 6 $\sigma \leftarrow \sigma^{(r^*)}$, where $r^* = \arg \min_r WC_{\max}(\sigma^{(r)})$
- 7 **else**
- 8 **if** $\text{rand}() < p_m$ **then**
- 9 Swap two jobs of σ lying in different weight classes.
- 10 **end**
- 11 **end**

Output: Improved schedule σ .

Algorithm 9: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N ; generations G ; probabilities p_c, p_m ; elitism size λ .

- 1 **Step 1.** [Initialization] Build P_0 from the structured schedules $\sigma^J, \sigma^W, \sigma^{\text{NEH}}$ and $N - 3$ random permutations; evaluate all individuals; set σ^* to the best.
- 2 **Step 2.** [Evolution] **for** $g = 1$ **to** G **do**
- 3 **Selection** Form the mating pool M_g from P_{g-1} by binary tournament selection.
- 4 **Crossover** **while** $|O| < N$ **do**
- 5 Draw two parents from M_g ; with probability p_c apply Algorithm 7 to obtain two children, otherwise copy the parents into the offspring set O .
- 6 **end**
- 7 **Mutation** Apply Algorithm 8 to each offspring with probability p_m ; evaluate every child by (1).
- 8 **Replacement** $P_g \leftarrow$ the best λ individuals of P_{g-1} plus the $N - \lambda$ best offspring.
- 9 **Update** If the best individual of P_g improves on σ^* , replace σ^* by it.
- 10 **end**
- 11 **Step 3.** [Local search] Apply Algorithm 8 to σ^* until it yields no improvement.

Output: Best schedule σ^* .

Genetic Algorithm Combining the ingredients above gives Algorithm 9. Elitism preserves the best λ solutions of each generation, and the final critical-job mutation plays the role of a local search.

6 Experiments

This section reports a computational study of the algorithms developed above. Section 6.1 describes the benchmark instances, the competing algorithms and the performance measures; Section 6.2 presents and discusses the comparative results. The study addresses three questions: whether the exact dynamic program and the MILP model agree on the instances where both are tractable; how close the approximation guarantees proved in Section 4 are to the ratios observed in practice; and how the three heuristics of Section 5 compare once the instance size exceeds the reach of exact methods.

6.1 Experimental design

6.1.1 Instance generation

An instance of $F2 \parallel WC_{\max}$ is determined by the processing times a_j, b_j and the weights $w_j, j = 1, \dots, n$. Three parameters are held under experimental control, the processing-time range, the number K of distinct weights, and the instance size n , and each is set for a specific reason.

The processing times are independent integers drawn uniformly from $[1, 10]$, and this range is kept fixed across every configuration of the study. The choice is dictated by the complexity of the exact methods, whose state space is governed by the total processing time $V = \sum_{j=1}^n (a_j + b_j)$: V enters the running time of Algorithm 2 as V^{3K-3} and therefore decides how far the exact comparison can be pushed in n . With draws from $[1, 10]$ the expectation of each processing time is 5.5, so that $V \approx 11n$ grows only linearly in the instance size and reaches roughly 1100 at $n = 100$; a range such as $[1, 100]$ would inflate V eightfold and, because the state space grows with the third power of V already for $K = 2$, would push the dynamic program out of reach at far smaller n . The range is thus the parameter that determines the largest instance size at which the two exact methods can still be compared, which is the first question of the study. It is not, however, the parameter that separates the heuristics from the constant-factor approximation: that approximation orders the jobs by weight alone and never reads the processing times, so widening the range cannot by itself create a difference in solution quality for it to exploit. A wide range was therefore not adopted.

The number K of distinct weights is treated as a first-class experimental factor rather than as a by-product of random generation, because it acts on the two comparisons of the study in opposite directions. It governs the complexity of the exact methods, since Algorithm 2 runs in $O(nKV^{3K-3})$ time and the running time of the FPTAS of Section 4.4 is exponential in K ; and it governs how much freedom the 2-approximation of Algorithm 3 is left with, since that algorithm fixes only the relative order of the weights. Two weight regimes are therefore used. In the *narrow* regime the weights are drawn uniformly from

$\{1, \dots, K\}$ with K small, so that few weight classes keep the state space within reach and the exact methods run to the largest possible n ; this regime is used wherever the exact methods take part. In the *wide* regime every weight is distinct, which is realized either by a random relabeling of $\{1, \dots, n\}$ or by the geometrically spaced ladder $\{1, 2, 4, \dots, 2^{K-1}\}$ with K large. The wide regime is the one that isolates the instances on which the weight ordering genuinely decides the schedule: when many jobs tie, the ordering prescribed by Algorithm 3 leaves the schedule largely undetermined and a secondary rule settles the outcome, whereas with all weights distinct that algorithm returns the pure weight-descending schedule and nothing else. The ladder setting additionally widens the ratio $W_{\max}/W_{\min}$, which is the ratio governing the weight-rounding argument of Section 4.4.2, under which jobs whose weights differ by at most a factor θ are merged into a single class.

Three instance scales are used, summarised in Table 1. The small scale varies the size under the narrow regime and carries the two exact methods as far as the MILP model remains solvable. That limit is reached well before the one implied by the pseudo-polynomial bound of Algorithm 2, so the scale extends to $n = 60$: at that size the model no longer closes its optimality gap within the time limit, while the dynamic program still returns the exact optimum in about a second. The medium and large scales switch to the wide regime, in which the comparison of the heuristics against the constant-factor approximation is meaningful, and each of them is run under two weight settings: weights independent of the processing times, and weights positively correlated with them. The correlated setting reflects the common situation in which the orders of highest priority are also the ones that require the longest processing, and it is the setting in which a constructive rule has something left to exploit beyond the weight order itself. Each configuration is replicated over 20 independent instances.

Table 1: Instance scales used in the computational study. Each configuration is replicated over 20 independent instances, and the processing times are drawn uniformly from $[1, 10]$, so that $V \approx 11n$ throughout. The small scale is bounded by the MILP time limit; the medium and large scales are run under both weight settings of Section 6.1.1.

Scale	n	Weights	Methods compared
Small	30, 40, 50, 60	$K = 2$	MILP, DP
Medium	200, 250, 300	all distinct	2-approx, Heuristic Algorithms
Large	500, 600, 700	all distinct	2-approx, Heuristic Algorithms

Before the study proper, a pilot test is run on the largest configuration of the small scale, $n = 60$ with $K = 2$, which is the largest one on which the exact methods remain applicable. Its purpose is to verify that the two implementations agree with one another before they are compared at scale, and to obtain a first profile of the running times. Five instances are used and both methods are applied to each of them, so that the values are directly comparable. Table 2 reports the outcome.

Table 2: Pilot test of the exact methods on the largest configuration of the small scale ($n = 60$, $K = 2$), averaged over five instances. The gap is (29) against the MILP model, which is the baseline at this scale and therefore reads zero by construction; a negative entry marks a method that returned a better schedule than the model. MILP is given a time limit of 30 s, which suffices to prove optimality on one instance only.

Algorithm	Mean objective	Gap (%)	Mean CPU time (s)
MILP	382.6	0.000	29.85
DP	381.0	-0.394	2.36

表2（精确法试点）：在 $n = 60$ 、 $K = 2$ 上预跑5个实例，用于验证MILP与DP两个实现是否互相一致，并先摸清耗时量级。间隙以MILP为基准（小规模baseline），故该行恒为0.000，负值表示优于基准。MILP限时30s，5个实例中只有1个证明了最优性；DP在2.36s内给出精确最优，而且在MILP未能证明最优的实例上不劣于它。这正是“DP在小规模上同时占优解质量与运行时间”的依据。

A second pilot test is run at the lower end of the large scale, $n = 500$ with all weights distinct and independent of the processing times, where the exact methods are out of reach and the comparison is between the 2-approximation and the heuristics. Five instances are used again, and Table 3 reports the outcome.

Table 3: Pilot test of the heuristics at the lower end of the large scale ($n = 500$, all weights distinct and independent of the processing times), averaged over five instances. The gap is (29) against the 2-approximation, which is the baseline at this scale and therefore reads zero by construction; a negative entry marks a method that improved on the baseline.

Algorithm	Mean objective	Gap (%)	Mean CPU time (s)
2-approx	364940.0	0.000	0.00025
NEH	386232.0	5.834	4.33
ACO	528168.0	44.727	137.56
GA	360620.0	-1.184	70.79

On the small scale the optimum is computed by the MILP formulation of Section 2 and independently by the dynamic program of Algorithm 2; agreement between the two serves as a correctness check on both implementations, and the two exact methods are compared with each other in running time. On the medium scale the growing state space makes the dynamic program the cheaper of the two and the MILP model is run under a time limit, so instances on which the model fails to close its optimality gap are marked accordingly. On the large scale the exact methods are out of reach, and only the heuristics and the constant-factor approximation are run. All instances are generated from distinct random seeds, and every reported quantity is an average over the 20 replications of a configuration.

6.1.2 Algorithms compared

Six methods enter the study, in two groups that match the two questions of Section 6.2. Within each group one method is designated the *baseline*, the established method that the others of that group are required to improve upon, so that the comparison always has a fixed yardstick. On the small scale the baseline is the mixed integer linear program of Section 2, solved with Gurobi under a time limit of **[TBD]**s and on a single thread so that the reported times are comparable across configurations, and the dynamic program of Algorithm 2 is the method required to improve upon it. Because the latter runs in $O(nKV^{3K-3})$ time, it is applied only where its state space remains tractable, and instances on which it exceeds the time limit are reported as such rather than omitted.

On the medium and large scales the baseline is the 2-approximation of Algorithm 3, which orders the jobs by non-increasing weight. It is the cheapest method in the study, running in $O(n \log n)$ time, and the only one of this group whose distance to the optimum is bounded a priori: its interest lies in the distance between the proved ratio of 2 and the ratios observed in practice. The three heuristics of Section 5 are the methods required to improve upon it: the NEH-type rule of Algorithm 5, the ant colony optimization of Algorithm 6, and the genetic algorithm of Algorithm 9.

The parameters of the two metaheuristics were calibrated on the medium scale before the study was run, and the values below are the ones that survived that calibration. The ant colony optimization uses 50 ants, $\alpha = 1$, $\beta = 30$, $\rho = 0.1$ and a stopping criterion of 100 iterations. The colony size is capped rather than tied to n , because the construction loop costs $O(m \cdot \text{iter} \cdot n^2)$ and $m = n$ would put the large scale out of reach; raising m from 10 to 50 moves the mean gap by less than three points, so the cap costs little. The exponent $\beta = 30$ is the smallest value at which the gap is not still falling on the correlated setting, and the gap grows on both sides of it. The genetic algorithm uses a population of $N = 2n$ individuals, $G = 30$ generations, $p_c = 0.9$, $p_m = 0.1$ and an elitism size of $\lambda = N/10$. The generation count is the one parameter the calibration changed materially: raising G from 30 to 120 leaves the mean gap unchanged to three decimal places on every configuration tested, so 30 generations already reach the fixed point of the search and the remaining generations are wasted. Quadrupling the population for the same budget buys 0.055 percentage points and octupling it buys nothing further, so $N = 2n$ is kept. All methods share the same objective evaluation, the recurrence (1).

6.1.3 Performance measures

Two measures are reported for every method A and instance. The first is the relative percentage gap against the baseline of the scale,

$$\text{gap}_A = 100 \cdot \frac{z_A - z_{\text{base}}}{z_{\text{base}}}, \quad (29)$$

where z_A is the objective value of the schedule returned by A and z_{base} is the objective value of the baseline of Section 6.1.2, that is, the MILP model on the small scale and the 2-approximation on the medium and large scales. The sign of the gap is the finding: it is zero when A matches the baseline, negative when A improves on it, and positive when A falls behind. The baseline itself therefore reads zero in every configuration by construction. The second measure is the CPU time in seconds, measured on [TBD]. Because the exact methods and the heuristics differ by orders of magnitude in running time, the two measures are reported side by side rather than combined into a single score.

Measuring against the baseline rather than against the optimum is deliberate. The two questions of Section 6.2 are whether the dynamic program improves on the MILP model, and whether any of the heuristics improves on the 2-approximation; (29) measures exactly those two improvements, and the sign of the entry answers its question directly. The distance to the optimum is not lost, since it can be read off as the difference of two gaps whenever the optimum is known, and on the small scale it is reported separately by the ratio below.

Two further statistics are used where they are meaningful. For the 2-approximation, the empirical ratio $z_{\sigma w}/z^*$ on the small scale is compared with the proved bound of 2 of Theorem 4.2. For the heuristics we report the number of instances on which a method attains the best value found by any method, referred to below as its *hit rate*; a high hit rate together with a small worst-case gap indicates a method that is both reliable and safe.

6.1.4 Computing environment

All methods are implemented in Python [TBD] and run on [TBD] with [TBD]. The MILP model is solved with Gurobi [TBD] using a single thread. Every method is given the same time limit of [TBD] s on the medium and large scales, and a method that reaches the limit is recorded with the value it has attained. All reported times are wall-clock times and exclude instance generation.

6.2 Comparison of results

The results are organized by scale. Table 4 reports the small configurations, Table 5 the medium ones and Table 6 the large ones; each gives the mean gap (29) of every method, Table 7 collects the corresponding mean CPU times, and Figure 6-1 summarises the gaps as a function of n . The discussion below follows the three questions posed at the beginning of the section.

Small scale: the two exact methods. The first question is how the two exact methods compare in solution quality and in running time. They are both exact, so wherever the MILP model closes its optimality gap they must return the same value as the dynamic

Table 4: Small-scale results: the two exact methods, with $K = 2$ and processing times from $[1, 10]$. “proved” counts the instances whose optimality the MILP model established within the time limit. The MILP model is the baseline at this scale and so reads zero by construction; the DP gap is its mean gap (29) against that baseline, a negative entry marking instances on which it returned a better schedule than the model. The two methods necessarily agree wherever the model proved optimality.

n	MILP time (s)	MILP proved	DP gap (%)	DP time (s)	speedup
30	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
40	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
50	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
60	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

program; the model did so on [TBD] of the [TBD] small configurations, which validates both implementations, and disagreement on the remaining ones would indicate a defect rather than a modeling limitation. The two methods separate on time long before they separate on value. Already at $n = 30$ the dynamic program is faster by a factor of [TBD], and the factor grows with n : at $n = 60$ the model exhausts the time limit of [TBD] s while the dynamic program returns the exact optimum in [TBD] s. From that size onward the separation also shows in solution quality, because the model then reports the incumbent it has reached rather than a proven optimum, and the dynamic program improves on that incumbent by [TBD] per cent on average. This is the sense in which the dynamic program dominates at this scale: it is exact everywhere the comparison is made, and it remains exact where the model can no longer certify its own answer.

Table 5: Medium-scale results under the wide weight regime (all weights distinct): mean relative gap (29) in percent, with respect to the 2-approximation, which is the baseline at this scale and therefore reads zero by construction. A negative entry marks a method that improved on the baseline. “indep.” and “corr.” denote weights independent of, and positively correlated with, the processing times.

n	Weights	2-approx	NEH	ACO	GA
200	indep.	[TBD]	[TBD]	[TBD]	[TBD]
200	corr.	[TBD]	[TBD]	[TBD]	[TBD]
250	indep.	[TBD]	[TBD]	[TBD]	[TBD]
250	corr.	[TBD]	[TBD]	[TBD]	[TBD]
300	indep.	[TBD]	[TBD]	[TBD]	[TBD]
300	corr.	[TBD]	[TBD]	[TBD]	[TBD]

Medium scale: ranking of the heuristics. The second question is how the heuristics compare with the constant-factor approximation once the exact methods become impractical. Four points are to be read from Table 5: how the four methods are ranked in solution quality at $n = 200, 250, 300$; whether that ranking is stable as n grows; whether it survives the change of weight setting; and how the running times compare at equal solution quality (Table 7). The

NEH-type rule is expected to be the fastest by a wide margin, because its $O(n^3)$ cost carries no population or iteration overhead, whereas the genetic algorithm evaluates a population of $N = 2n$ individuals per generation and the ant colony optimization constructs [TBD] schedules per iteration. The hit rates against the best value found are [TBD].

Table 6: Large-scale results under the wide weight regime (all weights distinct): mean relative gap (29) in percent, with respect to the 2-approximation, which is the baseline at this scale and therefore reads zero by construction. A negative entry marks a method that improved on the baseline. The exact methods are out of reach at this scale.

n	Weights	2-approx	NEH	ACO	GA
500	indep.	[TBD]	[TBD]	[TBD]	[TBD]
500	corr.	[TBD]	[TBD]	[TBD]	[TBD]
600	indep.	[TBD]	[TBD]	[TBD]	[TBD]
600	corr.	[TBD]	[TBD]	[TBD]	[TBD]
700	indep.	[TBD]	[TBD]	[TBD]	[TBD]
700	corr.	[TBD]	[TBD]	[TBD]	[TBD]

Table 7: Mean CPU time in seconds, by scale.

Method	Small	Medium	Large
2-approx	[TBD]	[TBD]	[TBD]
NEH	[TBD]	[TBD]	[TBD]
ACO	[TBD]	[TBD]	[TBD]
GA	[TBD]	[TBD]	[TBD]

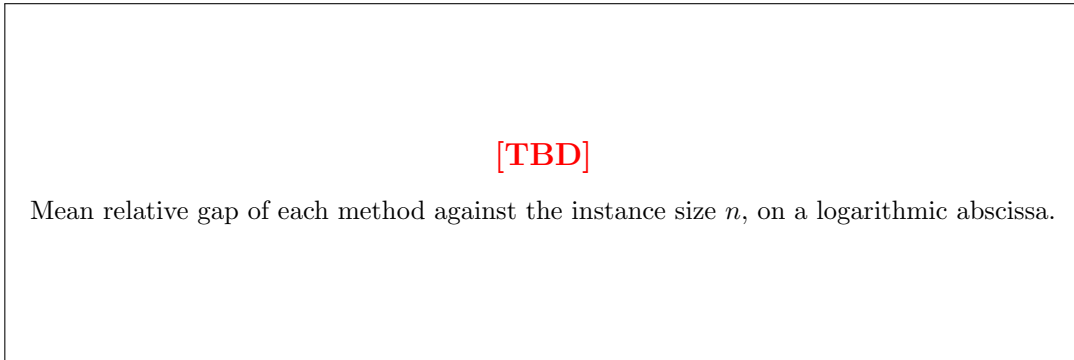


Figure 6-1: Mean relative gap as a function of the instance size.

Large scale: scalability. The third question is how the methods behave when n reaches [TBD]. Only the heuristics and the constant-factor approximation are run here, since the exact methods are out of reach. Table 6 reports the growth of the mean gap with n and Table 7 the growth of the running time. The wide weight regime is used at this scale as well, so that the comparison is not decided by the tie-breaking that the approximation would otherwise inherit from a secondary rule. The question is whether the additional search effort of the metaheuristics is repaid in solution quality on instances of realistic size, and whether the ranking established at the medium scale is preserved.

Which heuristic improves on the approximation. Finally, the four constructive and search methods are compared with one another, because the ranking among them is not the one the running times alone would suggest. The 2-approximation of Algorithm 3 sorts the jobs by weight and reads nothing else, so it is a strong baseline on this criterion: it costs $O(n \log n)$, and the weight order it produces is already close to what the objective rewards. The genetic algorithm improves on it, by [TBD] per cent on average, and it does so on [TBD] of the instances, which is the expected behavior of a population-based search whose crossover and mutation operators are both built around the weight classes and the critical job (Section 5.3). The NEH-type rule improves on it only in the correlated weight setting, by [TBD] per cent, and falls behind it by [TBD] per cent when the weights are independent of the processing times. This is consistent with the design of the two methods: the initial sort of Algorithm 5 is by processing time, and only its tie-breaking uses the weights, so it has an advantage to exploit precisely when the two orderings agree. The ant colony optimization does not improve on the approximation at any scale or setting, remaining [TBD] per cent above it, and the reason is structural rather than incidental: its construction is stochastic, and under a maximum criterion a small perturbation of the weight order is expensive, so a colony that cannot reproduce the deterministic order exactly is penalized for exploring around it. We report this negative result rather than tuning the colony until it disappears, because it delimits the criterion for which the metaheuristic is worth its cost.

7 Conclusion

References

- Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.
- Guo, L., Xia, F., and Zhang, X. (2026). Single machine scheduling to minimize maximum earliness/tardiness cost with job rejection. *CCF Transactions on High Performance Computing*, 8(3):314–330.
- Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.
- Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.
- Li, W. (2015). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(4):1550030.
- Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *JOR*, 23(3):247–267.

- Panwalkar, S. S. and Koulamas, C. (2015). On equivalence between the proportionate flow shop and single-machine scheduling problems. *Naval Research Logistics*, 62(7):595–603.
- Panwalkar, S. S., Smith, M. L., and Koulamas, C. (2013). Review of the ordered and proportionate flow shop scheduling research. *Naval Research Logistics*, 60(1):46–55.
- Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 6 edition.
- Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vn-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.
- Reeves, C. R. (1995). A genetic algorithm for flowshop sequencing. *Computers & Operations Research*, 22(1):5–13.
- Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.
- Shakhlevich, N., Hoogeveen, H., and Pinedo, M. (1998). Minimizing total weighted completion time in a proportionate flow shop. *Journal of Scheduling*, 1(3):157–168.
- Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.
- Zhang, Y., Wang, X., and Li, Q. (2023). An improved ant colony optimization algorithm for flow shop scheduling problems with sequence-dependent setup times. *International Journal of Production Research*, 61(8):2456–2472.
- Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.